



CampusFind: Full-Stack Campus Lost and Found Mobile Application

Unit Code and Title	ICT205 Mobile Application Development
Assessment Name & No.	Assessment 3 – Full-Stack Industry Case Study
Teaching Period / Year	2026
Lecturer / Unit Coordinator	Dr Ibrahim Alani
Student Full Name	Wais Ahmed
Student ID	CIHE231774
Submission Date	04/10/2026
Submission Type	Group

Team Members

Student Name	Student ID	Contribution (%)	Responsibilities for the Assessment
Wais Ahmed	CIHE231774	60%	Implementation of Firebase, CampusFind app and release. Also implemented main app features.
Salman Hassan Mostofa SHOSHE	CIHE231333	40%	Tested app and ran key aspects. Assisted with documentation and report.

Declaration

By submitting this assessment, I/we confirm that this work is my/our own and complies with the CIHE Academic Integrity Policy. Any use of AI or other tools has followed the assessment instructions and has been fully acknowledged in the AI/Tool Use Declaration Table below. I/we understand that inappropriate or unacknowledged use of AI, or submission of work that is not my/our own, may constitute academic misconduct. I/we also confirm that this assessment has not been previously submitted unless approved, and I/we consent to its use for standard academic integrity and quality assurance processes.

Student(s) Signature: wais

Date: 04/10/2026

GenAI Tool	Description
ChatGpt	Used for troubleshooting, technical information, report editing, project evidence documentation, brainstorming and user interaction. All work produced was checked and approved by team members.

Problem Definition

Universities involve many people within the university including students, staff and visitors that transit through buildings including lecture rooms, staff rooms, labs, canteens and other shared space. In these situations, students often lose items such as bags, books, ID cards, headphones, wallets, keys etc.

Lost and found items are frequently reported via many sources including reception, security, social media, student society or word of mouth. These reports are often not centralized, resulting in a situation where the owner of a lost item may not be aware of where their report has been submitted.

For this project the key problem was that there was no simple app for users within a university to report a lost item, to report a found item, to search for lost and found items and to record that a found item has been returned or matched.

CampusFind was created to solve this problem. It offers a centralized mobile app for Android users. Only authenticated users can create and edit lost and found items. The primary user is a university student, but other users such as university employees, student services, campus maintenance and security may also use the app.

This project tackles a real issue that can exist on a university campus and satisfies the marks available for a remote backend and use of sensors in a mobile app.

One Drive Link: [Campus Find Project](#)

GitHub Link: [GitHub_Campus_Find](#)

Requirements Analysis

The functionality of CampusFind required a full user life cycle from login to item retrieval.

Users should be able to register and log in securely. **Google Sign-In** is supported via Firebase Authentication as well as email and password authentication.

Upon login there is the ability to create a Found or Lost item. A Found or Lost item has fields for title of the item, type, status, description, date, category and the Firebase UID for the user creating the item (Firebase, n.d.).

CampusFind also utilizes several device capabilities. A user can either take a picture with the camera or select a picture from the photo gallery. GPS location is also supported to allow for approximate location of when the item was lost or found.

There are separate screens for Found and Lost items. Items can be searched by description, category or title of the item. Items can also be filtered by status:

1. **Active**
2. **Matched**
3. **Returned**

The My Reports page will only show reports made by the currently signed in user. The owner of a report can choose to change or remove a report from within the Item Details page.

Usability, ease of navigation, visual display, data validation, user feedback, reactivity to user input and secure access to back-end data are all non-functional features of the app.

Constraints involve reliance on an internet connection for Firebase functionality; permission needed for GPS and camera access. The ability to choose images and view them locally is also currently a constraint as the images are not yet uploaded to Firebase Storage (Firebase, n.d.).

The final app features more than the minimum of five functionality screens required by the assignment. There is also meaningful sensor usage, remote data storage, user interaction, lists, forms, data validation and all these combined into an aesthetically displayed app.

Solution Design

The app is architected in layers; from the user interface to the application logic, Firebase and device functionality.

- **Flutter User Interface**
↓
- **Application Service Classes**
↓
- **Firebase Authentication + Cloud Firestore**
↓
- **Camera / Gallery + GPS Location**

The Flutter interface includes the following main screens:

- | | | |
|--------------------|--------------------------|------------------------|
| 1. Login | 4. Report an Item | 7. My Reports |
| 2. Register | 5. Lost Items | 8. Item Details |
| 3. Home | 6. Found Items | |

Authentication in the app is managed via Firebase. Registered users are stored and permission to use the app is managed through authentication. A listener is attached to the authentication state, and the user will see either the Login or Home screen based on their authentication state.

A remote database is provided via Cloud Firestore. There are two primary collections:

Users:

- | | | |
|----------------|--------------------|---------------------|
| 1. uid | 3. email | 5. photoUrl |
| 2. name | 4. provider | 6. createdAt |

Items:

- | | | |
|-----------------------|---------------------|----------------------|
| 1. title | 5. status | 9. latitude |
| 2. description | 6. createdBy | 10. longitude |
| 3. type | 7. eventDate | 11. imageUrl |
| 4. category | 8. createdAt | |

The **'createdBy'** value is the Firebase UID of the user that created the report. This is used to determine the owner of the report and to limit certain owner-only functions such as deleting or changing the status of the report (Firebase, n.d.).

When a report is created it undergoes validation before being saved to Firestore. Updates to a report and creation of new reports are observed via Firestore streams in My Reports, Lost Items and Found Items to provide real-time updates to the user.

The app has been designed to be simple and uniform across all screens. Lost reports are displayed in red and Found reports in green. Filters, status badges, text input fields and a back button aid in the user experience.

Technology Stack Justification and Implementation Overview

The app is architected in layers; from the user interface to the application logic, Firebase and device functionality.

Authentication in the app is managed via Firebase. Registered users are stored and permission to use the app is managed through authentication. A listener is attached to the authentication state and the user will see either the Login or Home screen based on their authentication state.

A remote database is provided via Cloud Firestore. There are two primary collections:

The **createdBy** value is the Firebase UID of the user that created the report. This is used to determine the owner of the report and to limit certain owner-only functions such as deleting or changing the status of the report.

When a report is created it undergoes validation before being saved to Firestore. Updates to a report and creation of new reports are observed via Firestore streams in My Reports, Lost Items and Found Items to provide real-time updates to the user.

The app has been designed to be simple and uniform across all screens. Lost reports are displayed in red and Found reports in green. Filters, status badges, text input fields and a back button aid in the user experience.

The final application includes:

- | | | |
|-----------------------------------|----------------------------|------------------------------------|
| 1. email/password authentication | 5. camera photo capture | 9. My Reports |
| 2. Google authentication | 6. gallery image selection | 10. Item Details |
| 3. Firestore user profiles | 7. GPS location capture | 11. owner status updates |
| 4. Lost and Found report creation | 8. search and filtering | 12. report deletion |
| | | 13. reusable navigation components |

There were a few implementation issues. One was setting up Google Sign-In. An Android SHA-1 certificate had to be created, and the fingerprint added to Firebase to set up the proper OAuth authentication.

A second issue was with user data in Firestore. Firebase Authentication supported authentication users, but an additional user document was needed in Firestore called users. This allows user data to be stored for use in the app.

A CampusFind AppBar was created to be reused throughout the app. All but the initial activity uses a back button to provide consistent navigation.

Currently images for items are not stored in the cloud. Images can be chosen and viewed, but Firebase Storage is not yet implemented.

Testing and Evaluation

CampusFind was tested by manual function testing, performance testing, code analysis and Android release builds.

Code analysis was achieved through:

- **account registration**
- **email/password login**
- **Google Sign-In**
- **logout**
- **navigation**
- **Firestore read/write operations**
- **Lost report creation**
- **Found report creation**
- **camera access**
- **gallery selection**
- **GPS permission and location capture**
- **search**
- **status filters**
- **My Reports**
- **status updates**
- **report deletion**

Function testing was achieved by testing from login to report creation, Firestore upload, browsing, Item Details and status update.

Performance testing was achieved by:

flutter run --profile

This enabled the app to be run in debug profile mode and enabled Flutter DevTools to be utilised for performance analysis (Flutter, n.d.).

An Android release APK was created by using:

flutter build apk --release

The APK created was installed and tested within the Android emulator.

An Android App Bundle was created by using:

flutter build appbundle --release

This demonstrates the app has moved from development to testing, profiling and release. The full process referenced in Week 11 is as follows: **Develop → Test → Profile → Build → Sign → Test Release → Upload → Publish → Monitor**.

Usability features of the app include search functionality, consistent colours, feedback, simple report functionality, easy to follow navigation and menus.

Issues found included:

- **No upload to Firebase Storage for images**
- **Relies on internet connection**
- **No automated unit, widget or integration tests**
- **No published app to the Play Store**

Future Enhancements

An initial set of notable enhancements could include support for Firebase Storage. This would enable some images to be uploaded and viewed across multiple devices rather than just being able to view them locally.

An alternative login option could be implemented using Microsoft or Outlook accounts.

Map and reverse geocoding could be implemented to give GPS points some context within the campus (Flutter, n.d.).

A matching feature could be implemented to match items reported to the Lost and Found according to date, description, location and category. A match could then be notified to the user.

Enhanced security could be implemented with production Firestore rules, greater security on permissions and a production Android signing keystore.

As your data grows, you can improve scalability by using more efficient queries, adding Firestore indexes, and supporting pagination.

Finally automated unit, widget and integration tests could be implemented in order to lower the risk of regression and increase overall reliability.

Conclusion

CampusFind is a mobile app that aims to address the issue of lost and found objects on campus.

CampusFind uses GPS, camera and gallery functionality, Firebase Authentication, Cloud Firestore and Flutter to allow a lost item to be reported and a found item to be searched for.

This project has used the entire mobile development cycle for the assignment including identifying the problem, analysing the requirements, designing a solution, coding the solution, backend implementation, integration of sensors, profiling and testing and finally preparing the app for release on Android.

The current implementation works and provides a good starting point from which to enhance with features including production release, additional authentication methods, automated matching, notifications, Firebase Storage and more.

References

Baseflow. Geolocator. Pub.dev. <https://pub.dev/packages/geolocator> Dart packages 2026.

Firebase. Google. Firebase Authentication. <https://firebase.google.com/docs/auth/> n.d.

Firebase. Google. Cloud Firestore. <https://firebase.google.com/docs/firestore> n.d.

Flutter. Guide to app architecture. <https://docs.flutter.dev/app-architecture/guide> Flutter Documentation n.d.

Flutter. Flutter architectural overview. <https://docs.flutter.dev/resources/architectural-overview> Flutter Documentation n.d.

Flutter. image_picker. Pub.dev. https://pub.dev/packages/image_picker Dart packages 2026.

OpenAI. ChatGPT [Large language model]. <https://chatgpt.com/> 2026.

Appendix – AI Governance

AI was used as an auxiliary in the creation and record of this project using **ChatGPT by OpenAI**

AI was used to:

- **Generate ideas**
- **Enhance the layout and language within the reports**
- **Clarify concepts within Flutter and Firebase**
- **Aid in the diagnosis and identification of any errors in code**
- **Aid in organising the evidence for testing and presentation**
- **Enhance the presentation and clarity of written work**

AI was used as an auxiliary only. The execution, testing, project validation, application usage, screenshots, implementation and final decision making was carried out by the students.